RDFIA 1 - ab

Julien Kordes

October 2024

Table des matières

1 Introduction **2**

2 Section 1 - Theoretical foundation **2**
2.1 Supervised Dataset 2
2.2 Network Architecture 2
2.3 Loss Function 4
2.4 Optimization algorithm 4

3 Implementation **10**
3.1 Forward and backward manuals 10
3.2 Simplification of the backward pass with torch.autograd 13
3.3 Simplification of the forward pass with torch.nn layers 15
3.4 Simplification of the SGD with torch.optim 17
3.5 MNIST Application 19

1 Introduction

This practical focuses on building a simple neural network but with different methods : it enables us to learn how to implement the backward and forward functions of a simple NN and then to replace them by more commonly used methods of the pytorch library. Indeed, understanding the role of each parameters of a NN and each function of a training procedure is fundamental in machine and deep learning.

2 Section 1 - Theoretical foundation

2.1 Supervised Dataset

Question 1

The train set is used to train the model's parameters. The model learns relations between the features' caracteristics and the corresponding outputs.

The val set is used to evaluate the model's performance during the training. It enabels us to see wether the model is able to generalize to data that he has not seen in the train set or not.

The test set is used at the end of the training. It contains data that the model has never seen and it's used to see the quality of the model and his ability to generalize to unseen data.

Question 2

The number of examples N has a big influence on the model's performances.

If it is a small dataset (N low), the training will be faster but the risk of overfitting will be higher and the model will have some difficulties to generalize to other data. On the contrary, a large datset (N high) will be longer to train but will also be more robust, more accurate and better in terms of generalization.

2.2 Network Architecture

Question 3

Adding activations functions between linear transformations in a Neural Network is important to introduce non-linearity on the model. Indeed, stacking many linear operations without introducing non-linearity results in a single linear operation. In

general, it helps the model to capture more complex caracteristics of the dataset in the deeper layers of the Neural Network.

Question 4

In the figure 1, $n_x = 2$, $n_y = 2$ and $n_h = 4$.

In practice, the size n_x corresponds to the dimension of the input data and the size n_y corresponds to the dimension of the output. Finaly, the size n_h is often chosen by testing many different sizes. Indeed there is no general rule: for a simple task the model will not need to have many different neurons in the hidden layer so n_h will be low but it will be higher for more complex tasks.

Question 5

The vector $\hat{y}$ represents the output of the model: it is the result of the transformations applied to the input x by the model. The vector y is the label vector: in a supervised dataset, this represents the target that the model should predict. The goal is to have the lowest difference between our prediction $\hat{y}$ and the target y. This difference is calculated by the loss that we try to minimize to improve our model's performances.

Question 6

In the context of classification, SoftMax is used as the output activation function because it can easily represent the probability of our feature to be in a class compared to another.

Question 7

The mathematical expressions are:

$$\hat{y} = SoftMax(\tilde{y})$$

$$\tilde{y} = W_y * h + b_y$$

$$h = Tanh(\tilde{h})$$

$$\tilde{h} = W_h * x + b_h$$

2.3 Loss Function

Question 8

In order to decrease the global loss function, the $\hat{y}_i$ must get closer to the value of y_i (for the regression tasks). Indeed, it is immediate that the MSE function is minimized when the value of $\hat{y}_i - y_i$ decreases. For the classification tasks, we want $\hat{y}_i$ to be close to 1 when $y_i = 1$ and to be close to 0 when $y_i = 0$. There is only one j where $y_j = 1$ and all the other $y_i = 0$. So we want $\hat{y}_j$ to be as close as possible to 1. This will minimize the loss for j but if we take into account the fact that the norm of $\hat{y}$ is 1, this will lead to every other $\hat{y}_{i i \neq j}$ being close to 0.

Question 9

MSE is better suited for regression because it directly measures the error between predicted and actual values, reflecting how close the predictions are to the true values. On the other hand, Cross-entropy is better for classification because it measures the difference between the predicted probability distribution and the true class distribution. Moreover it strongly penalizes wrong classifications as we saw on the previous question and encourages the model to do correct predictions with high-confidence.

2.4 Optimization algorithm

Question 10

Classic gradient descent offers strong stability, especially with convex loss functions, through deterministic and reproducible updates. Nonetheless, this stability can lead to being trapped in local minima for non-convex loss functions. Additionally, it is computationally expensive for large datasets, as each iteration requires evaluating the gradient across the entire dataset.

Mini-Batch Stochastic Gradient Descent (SGD) converges faster than classic gradient descent by updating model parameters with small, random subsets of data (mini-batches). This stochastic nature helps escape local minima and better explore the loss landscape, though it may introduce noise and oscillations.

Online Stochastic Gradient Descent allows for extremely rapid updates by processing one training example at a time. However if the learning rate is not carefully chosen, the highly noisy updates can lead to erratic convergence or even divergence.

In the general case, mini-batch stochastic gradient descent is the most balanced choice, combining the computational efficiency of classic gradient descent with the noise resilience and convergence speed of online SGD.

Question 11

The learning rate is very important to shape the learning process as it affects the convergence speed, the stability, and the model quality. Choosing the right learning rate often requires experimentation and can vary by problem.

A **higher learning rate** can speed up the convergence by making larger parameter updates, but if it is too high, it may cause issues like overshooting or divergence. On the other hand, a **lower learning rate** allows more careful exploration of the loss, potentially avoiding shallow minima but can lead to slower convergence or getting stuck.

Learning rate schedulers, adaptive methods like Adam, and batch size adjustments help optimize stability and convergence. Moreover techniques like grid and random search can help selecting the ideal learning rate for a specific task.

Question 12

The **naive method** calculates gradients independently for each parameter, leading to factorial complexity, $\mathcal{O}(n!)$ (n being the number of paramters), which becomes computationally very expensive as the number of layers increases.

On the other hand, **backpropagation** applies the chain rule to optimize gradient calculations (the gradient with respect to the output can be reused to calculate the gradient with respect to the input of a layer), reducing complexity to $\mathcal{O}(n)$. This makes backpropagation far more efficient and scalable, enabling practical training of deep neural networks.

Question 13

The network architecture needs to meet a few criteria for us to be able to use the backpropagation algorithm. First, each layer function, including the activation functions and the loss function, have to be differentiable with respect to their parameters. Then, the network architecture should possess a feedforward, sequential structure without loops or recurrent connections as backpropagation relies on the sequential flow of information from the input layer to the output layer.

Question 14

The **cross-entropy loss** is defined as $l(y, \hat{y}) = -\sum_{i=1}^{n_y} y_i \log \hat{y}_i$, where n_y is the total number of classes.

The **SoftMax function** is defined as $\text{SoftMax}(\tilde{y}_i) = \frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}}$. Moreover, the output of the SoftMax function is the input of the cross-entropy loss so this leads us to :

$$l(y, \hat{y}) = -\sum_{i=1}^{n_y} y_i \log \hat{y}_i = -\sum_{i=1}^{n_y} y_i \log \text{SoftMax}(\tilde{y}_i)$$

$$= -\sum_{i=1}^{n_y} y_i \log \frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}} = -\sum_{i=1}^{n_y} y_i \left[\log e^{\tilde{y}_i} - \log \sum_{j=1}^{n_y} e^{\tilde{y}_j}\right] = -\sum_{i=1}^{n_y} y_i \tilde{y}_i - y_i \log \left(\sum_{j=1}^{n_y} e^{\tilde{y}_j}\right)$$

$$= -\sum_{i=1}^{n_y} y_i \tilde{y}_i + \sum_{i=1}^{n_y} y_i \log \left(\sum_{j}^{n_y} e^{\tilde{y}_j}\right) = -\sum_{i=1}^{n_y} y_i \tilde{y}_i + \log \left(\sum_{j}^{n_y} e^{\tilde{y}_j}\right) \sum_{i=1}^{n_y} y_i$$

Moreover we know that $\sum_{i=1}^{n_y} y_i = 1$ so this gives us :

$$l(y, \hat{y}) = -\sum_{i=1}^{n_y} y_i \tilde{y}_i + \log \left(\sum_{i=1}^{n_y} e^{\tilde{y}_i}\right)$$

Question 15

$$\begin{aligned}
\frac{\partial l}{\partial \tilde{y}_i} &= -y_i + \frac{\frac{\partial}{\partial \tilde{y}_i}(\sum_{j=1}^{n_y} e^{\tilde{y}_j})}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}} \\
&= -y_i + \frac{e^{\tilde{y}_i}}{\sum_{j=1}^{n_y} e^{\tilde{y}_j}} \\
&= -y_i + \text{SoftMax}(\tilde{y})_i
\end{aligned}$$

Finally :

$$\nabla_{\tilde{y}} l = \begin{pmatrix} \frac{\partial l}{\partial \tilde{y}_1} \\ \vdots \\ \frac{\partial l}{\partial \tilde{y}_{n_y}} \end{pmatrix} = \hat{y} - y$$

Question 16

Calculation of $\nabla_{W_y} l$:

$$\frac{\partial l}{\partial W_{y,ij}} = \sum_k \frac{\partial l}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial W_{y,ij}}$$

$$\nabla_{W_y} l = \begin{pmatrix} \frac{\partial l}{\partial W_{y,11}} & \dots & \frac{\partial l}{\partial W_{y,1n_h}} \\ \vdots & \ddots & \vdots \\ \frac{\partial l}{\partial W_{y,n_y1}} & \dots & \frac{\partial l}{\partial W_{y,n_yn_h}} \end{pmatrix}$$

Firstly let's look at $\tilde{y}_k$:

$$\tilde{y}^T = h^T W_y^T + b_y^T$$

$$= \begin{pmatrix} h_1 & \dots & h_{n_h} \end{pmatrix} * \begin{pmatrix} W_{1,1} & \dots & W_{1,n_y} \\ \vdots & \ddots & \vdots \\ W_{n_h,1} & \dots & W_{n_h,n_y} \end{pmatrix} + \begin{pmatrix} b_1^y & \dots & b_{n_y}^y \end{pmatrix}$$

$$= \begin{pmatrix} \sum_{j=1}^{n_h} W_{y,j1} h_j + b_1^y & \sum_{j=1}^{n_h} W_{y,j2} h_j + b_2^y & \dots & \sum_{j=1}^{n_h} W_{y,jn_h} h_{k,j} + b_{n_h}^y \end{pmatrix}$$

$$= \left(\sum_{j=1}^{n_h} W_{y,jk} h_j + b_k^y \right), k \in [1, n_h] \text{ (Swap } j \text{ and } k \text{ to get } \tilde{y})$$

Calculation of the partial derivative of $\tilde{y}_k$ with respect to $W_{y,ij}$:

$$\frac{\partial \tilde{y}_k}{\partial W_{y,ij}} = \begin{cases} h_j & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}$$

Calculation of $\frac{\partial l}{\partial \tilde{y}_k}$:
From the (Question 15) :

$$\frac{\partial l}{\partial \tilde{y}_k} = -y_k + \text{SoftMax}(\tilde{y})_k = \hat{y}_k - y_k$$

This leads us to:

$$\frac{\partial l}{\partial W_{y,ij}} = \sum_{k=1}^{n_h} \frac{\partial l}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial W_{y,ij}} = \frac{\partial l}{\partial \tilde{y}_i} \frac{\partial \tilde{y}_i}{\partial W_{y,ij}} = (\hat{y}_i - y_i) h_j = (\nabla_{W_y} l)_{ij}$$

To conclude :

$$\nabla_{W_y} l = \begin{pmatrix} (\hat{y}_1 - y_1) h_1 & \dots & (\hat{y}_1 - y_1) h_{n_h} \\ \vdots & \ddots & \vdots \\ (\hat{y}_{n_y} - y_{n_y}) h_1 & \dots & (\hat{y}_{n_y} - y_{n_y}) h_{n_h} \end{pmatrix}$$

The calculation of the gradient with respect to the bias is the same and leads to :

$$\frac{\partial l}{\partial b_k^y} = \sum_{k=1}^{n_h} \frac{\partial l}{\partial \tilde{y}_k} \frac{\partial \tilde{y}_k}{\partial b_k^y} = \hat{y}_k - y_k$$

Question 17

- Using the chain rule we get :

$$\frac{\partial l}{\partial \tilde{h}_i} = \sum_k \frac{\partial l}{\partial h_k} \frac{\partial h_k}{\partial \tilde{h}_i}$$

Firstly:

$$\frac{\partial h_k}{\partial \tilde{h}_i} = \frac{\partial \tanh(\tilde{h}_k)}{\partial \tilde{h}_i} = \begin{cases} 1 - \tanh^2(\tilde{h}_i) = 1 - h_i^2 & \text{if } k = i \\ 0 & \text{otherwise} \end{cases}$$

Then, with previous questions' results we can write :

$$\frac{\partial l}{\partial h_k} = \sum_{j=1} \frac{\partial l}{\partial \tilde{y}_j} \frac{\partial \tilde{y}_j}{\partial h_k} = \sum_{j=1} (\hat{y}_j - y_j) W_{y,jk}$$

And finally,

$$\begin{aligned} \frac{\partial l}{\partial \tilde{h}_i} &= \sum_k \sum_j (\hat{y}_j - y_j) W_{y,jk} \times \begin{cases} 1 - h_i^2 & \text{if } k = i \\ 0 & \text{otherwise} \end{cases} \\ &= (1 - h_i^2) \left(\sum_j (\hat{y}_j - y_j) W_{y,ji} \right) \end{aligned}$$

To conclude, the gradient $\nabla_{\tilde{h}} l$ is the vector:

$$\nabla_{\tilde{h}} l = (1 - h^2)(\nabla_{\tilde{y}} l * W^y)$$

- Once again using the chain rule :

$$\frac{\partial l}{\partial W_{h,ij}} = \sum_k \frac{\partial l}{\partial \tilde{h}_k} \frac{\partial \tilde{h}_k}{\partial W_{h,ij}}$$

We know that $\frac{\partial l}{\partial h_k} = (1 - h_k^2)(\sum_j (\hat{y}_j - y_j) W_{y,jk})$

Let's calculate $\frac{\partial \tilde{h}_k}{\partial W_{h,ij}}$ knowing that $\tilde{h}_k = \sum_{j=1}^{n_x} W_{h,kj} x_j + b_k^h$

$$\frac{\partial \tilde{h}_k}{\partial W_{i,j}^h} = \begin{cases} x_j & \text{if } k = i \\ 0 & \text{otherwise} \end{cases}$$

To conclude,

$$\begin{aligned} \frac{\partial l}{\partial W_{i,j}^h} &= \sum_k (1 - h_k^2)(\sum_j (\hat{y}_j - y_j) W_{y,jk}) \times \begin{cases} x_j & \text{if } k = i \\ 0 & \text{otherwise} \end{cases} \\ &= (1 - h_i^2)(\sum_j (\hat{y}_j - y_j) W_{y,ji}) x_j \end{aligned}$$

And then,

$$\nabla_{W_h} l = \nabla_{\tilde{h}} l * x$$

- Finally the calculations are the same for $\nabla_{b_h} l$ and lead to:

$$\nabla_{b_h} l = \nabla_{\tilde{h}} l$$

3 Implementation

This section studies the performances of the neural network described in the previous section but using different types of implementation.

3.1 Forward and backward manuals

The FIGURE 1 and FIGURE 2 show the influence of the learning rate on the performance of our model using forward and backward manuals. Choosing a very small learning rate prevents the convergence of the model even after 400 iterations. And we can see that the best performance is achieved with **lr = 0.1** where the accuracy reaches its maximum after only **50 iterations**. These big learning rates add more noise to the model according to the curves but help the accuracy to grow faster.

For the batch size, we can see on the textscFigure 3 and on the FIGURE 4 that a smaller batch size helps the model to converge faster. The aspect of the curves also shows that a larger batch size gives more stability to the model (the accuracy increases more regularly) but it takes more time to converge.

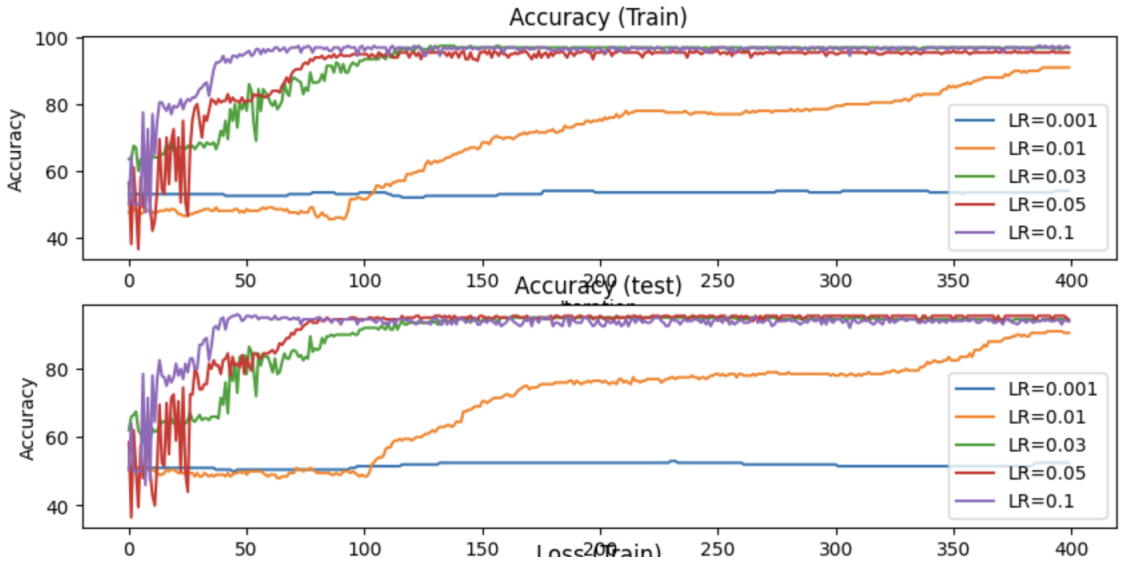


Figure 1: Influence of learning rate on the accuracy of the first method using forward and backward manuals

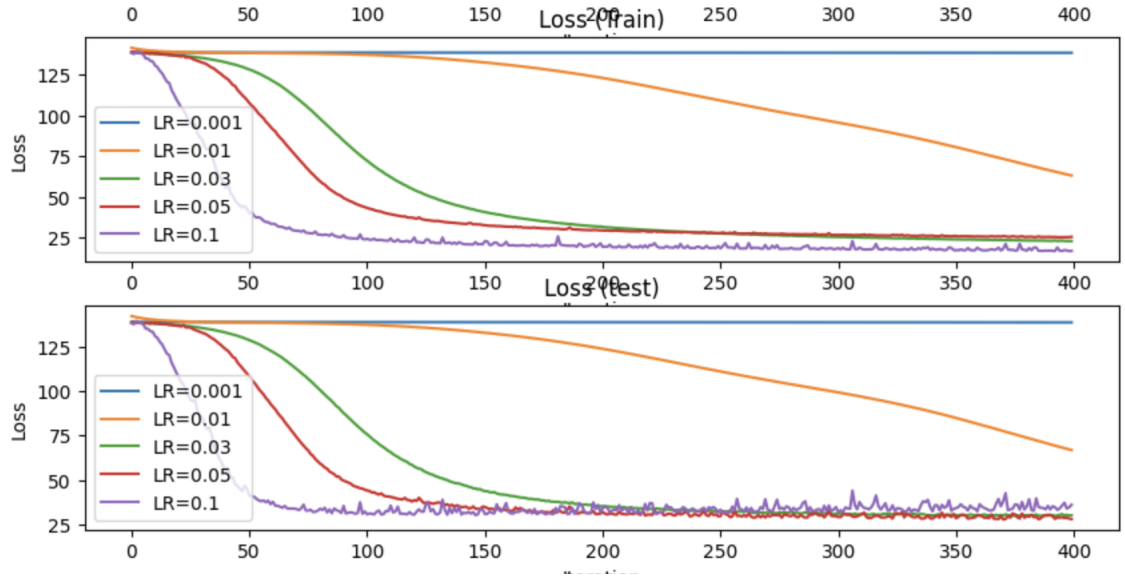


Figure 2: Influence of learning rate on the loss of the first method using forward and backward manuals

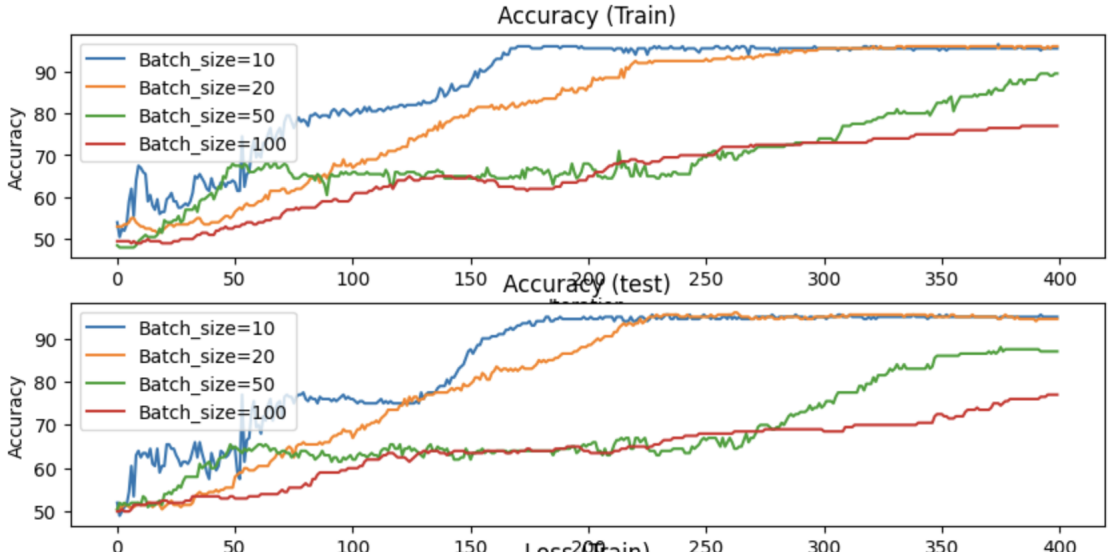


Figure 3: Influence of the batch size on the accuracy of the first method using forward and backward manuals

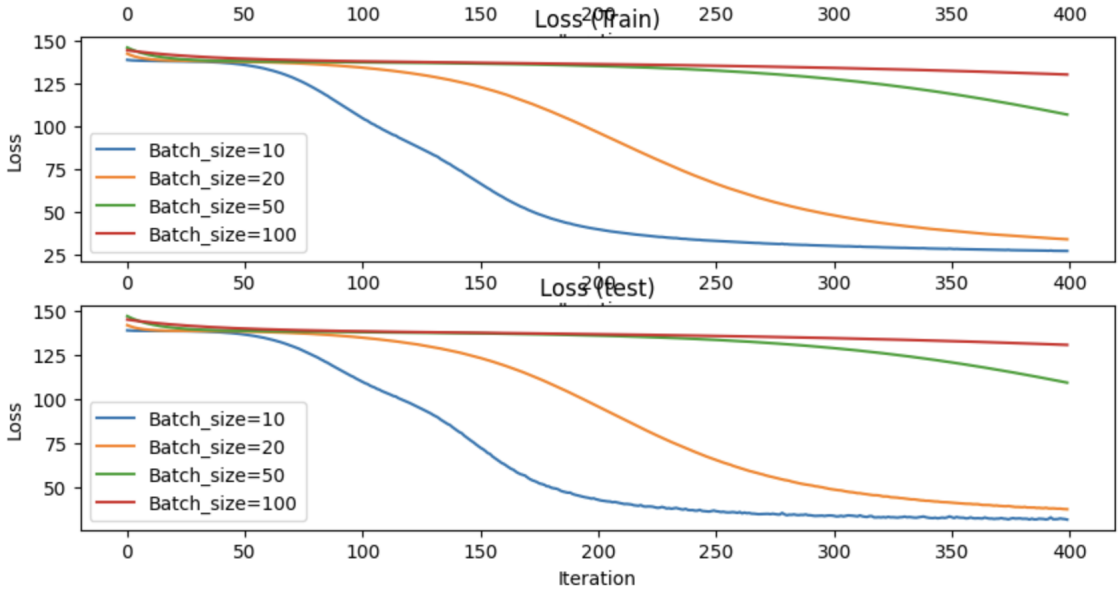


Figure 4: Influence of the batch size on the loss of the first method using forward and backward manuals

3.2 Simplification of the backward pass with torch.autograd

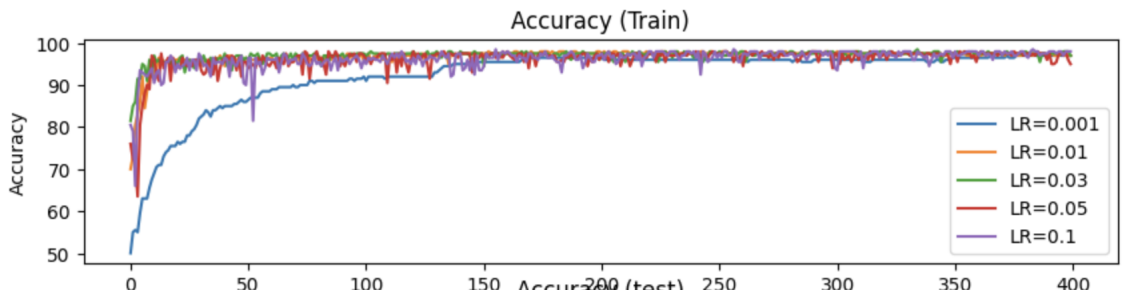


Figure 5: Influence of the learning rate on the accuracy of the second method using torch.autograd for the backward

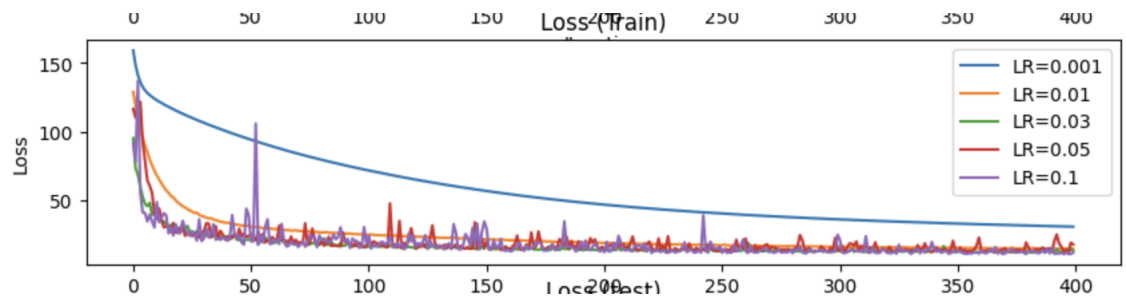


Figure 6: Influence of the learning rate on the loss of the second method using torch.autograd for the backward

The first thing that we can say when using torch.autograd in stead of a manual backward is that it is way faster to compute. Indeed, this method only took **38 secondes** to get 400 iterations in stead of **12 minutes** for the manual method.

Moreover we can see on the Figure 7 and Figure 8 that the effect of the learning rate is pretty much the same on the accuracy and loss (I only printed the training one since it is very similar to the test).

However, it converges way faster in general since it only takes about less than 10 iterations to reach the maximum accuracy for a learning rate higher than 0.03. The stability of the model is also improved especially at the beginning of the learning where we saw way more fluctuations for the previous method when the learning rate was high.

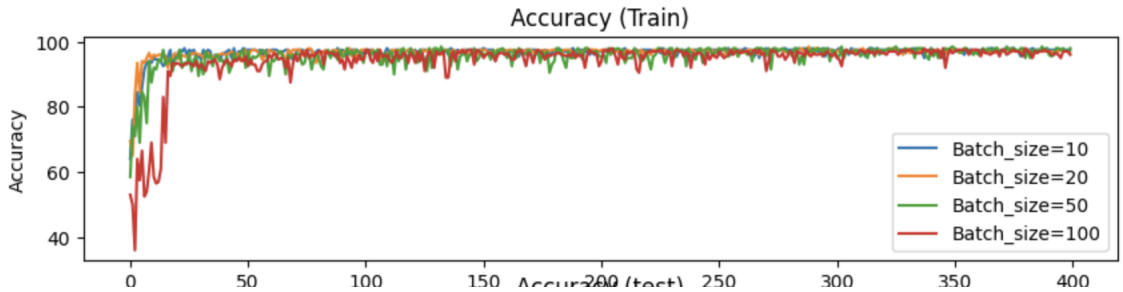


Figure 7: Influence of the batch size on the accuracy of the second method using torch.autograd for the backward

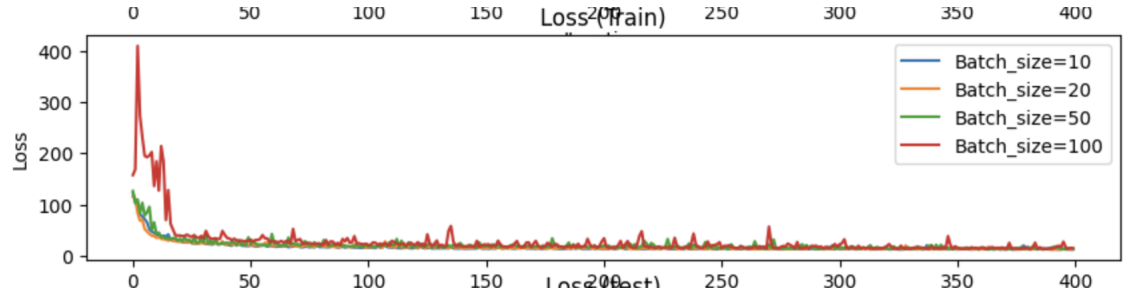


Figure 8: Influence of the batch size on the loss of the second method using torch.autograd for the backward

The conclusions are the same concerning the batch size that doesn't change much from the previous method except that the convergence and stability of the model is increased thanks to the torch.autograd.

3.3 Simplification of the forward pass with torch.nn layers

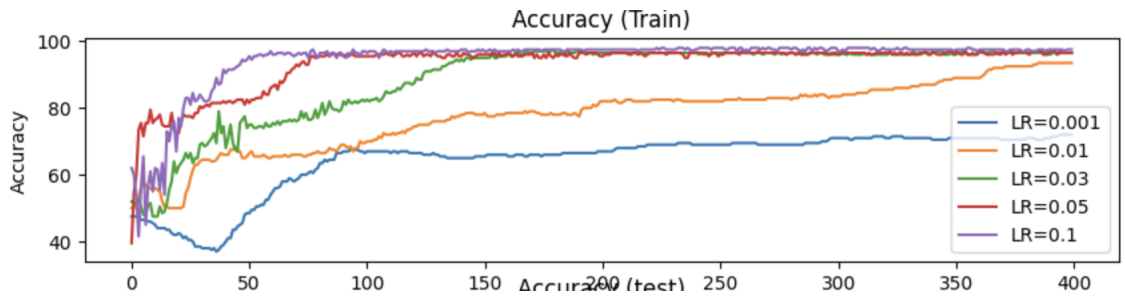


Figure 9: Influence of the learning rate on the accuracy of the third method using torch.autograd and torch.nn layers

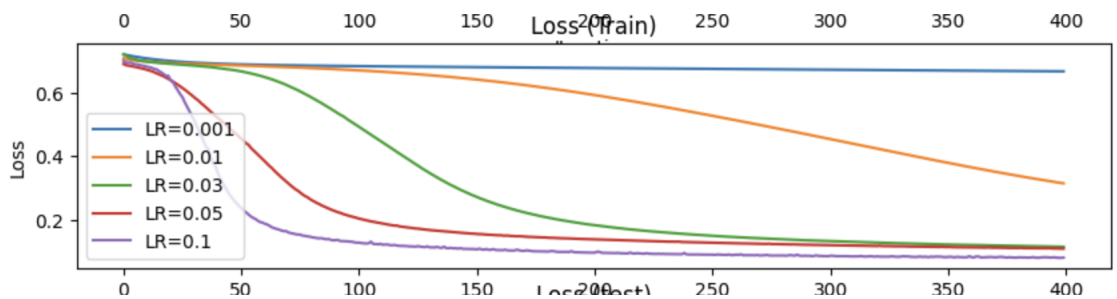


Figure 10: Influence of the learning rate on the loss of the third method using torch.autograd and torch.nn layers

Here the curves obtained look pretty much like the first method. Even though it is way faster to compute (only a few seconds instead of 10 minutes), the model faces pretty much the same issues for the choice of learning rate and batch size.

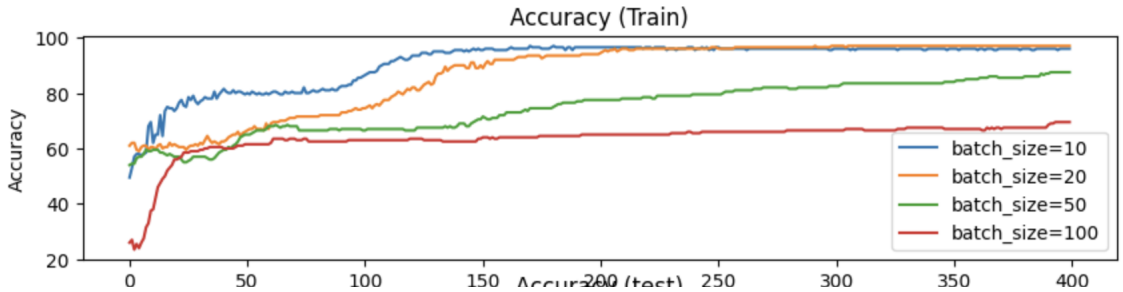


Figure 11: Influence of the batch size on the accuracy of the third method using torch.autograd and torch.nn layers

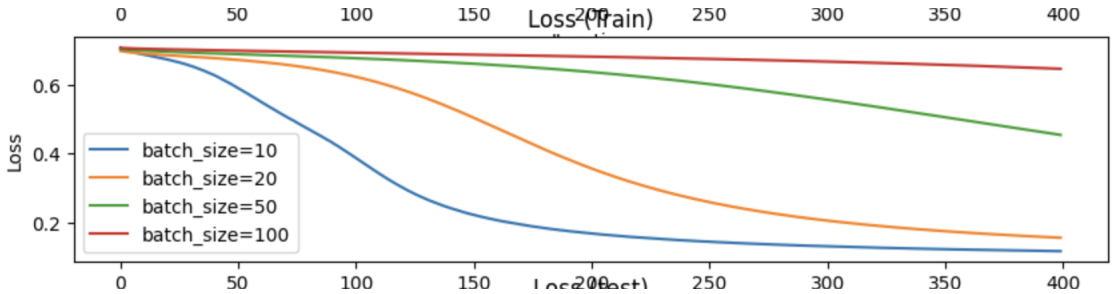


Figure 12: Influence of the batch size on the loss of the third method using torch.autograd and torch.nn layers

3.4 Simplification of the SGD with torch.optim

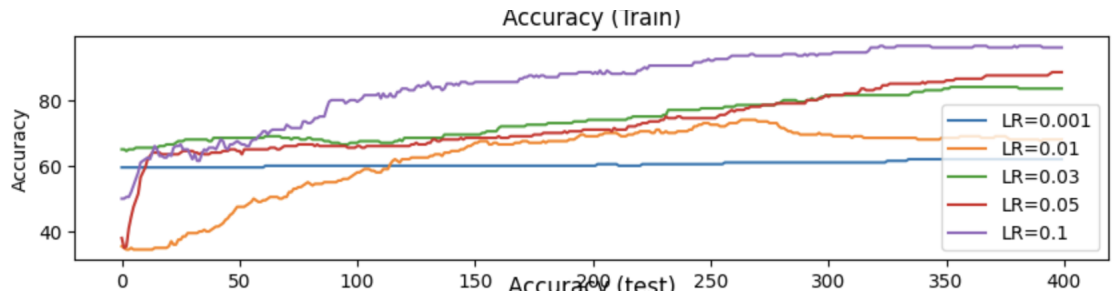


Figure 13: Influence of the learning rate on the accuracy of the fourth method using torch.optim

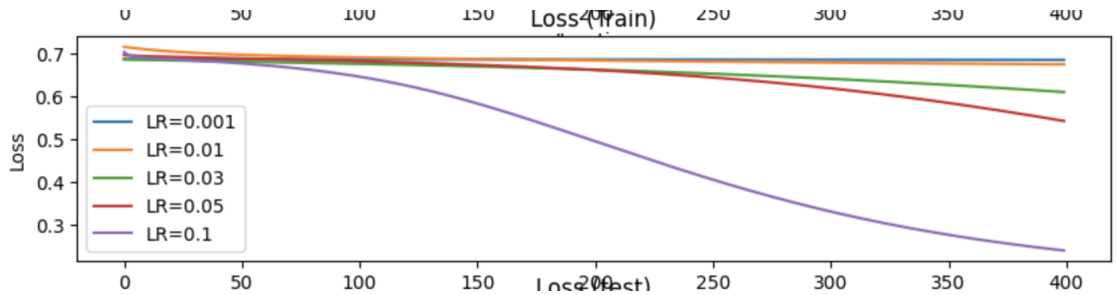


Figure 14: Influence of the learning rate on the loss of the third method using torch.optim

As we can see on the Figure 15 and Figure 16, using the simplification of the SGD contained in the torch.optim library decreases the performance of the model. The computation is faster but the learning is not precise enough and the loss function takes way more iterations to start decreasing.

Concerning the influence of the learning rate and the batch size it stays the same regardless of the method we use.

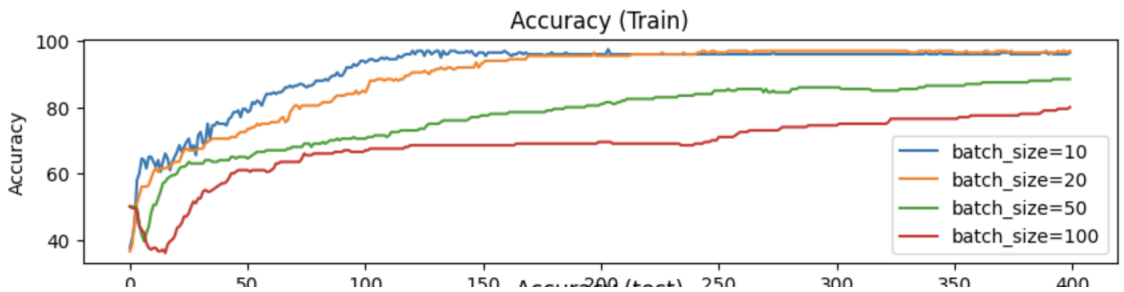


Figure 15: Influence of the batch size on the accuracy of the fourth method using torch.optim

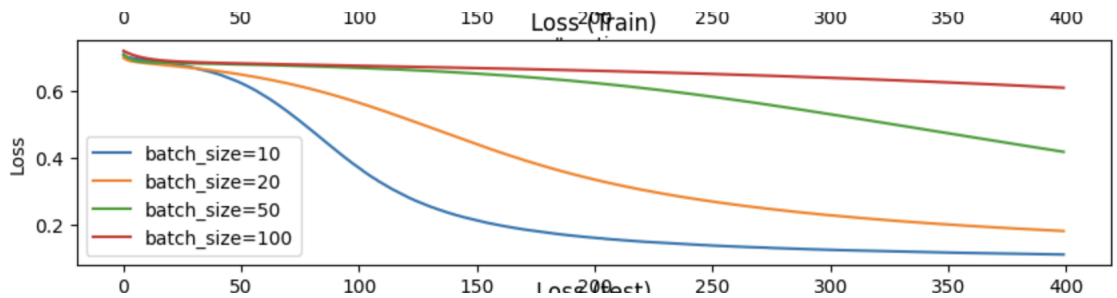
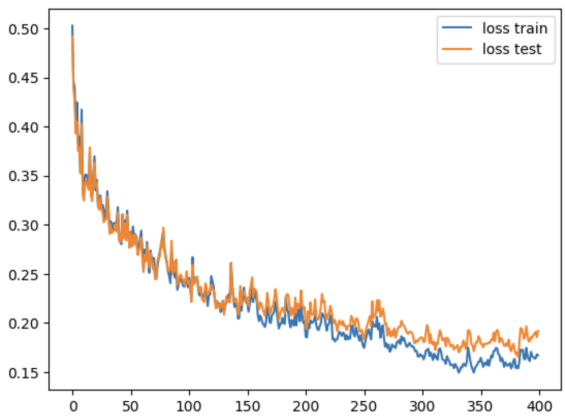


Figure 16: Influence of the batch size on the loss of the third method using torch.optim

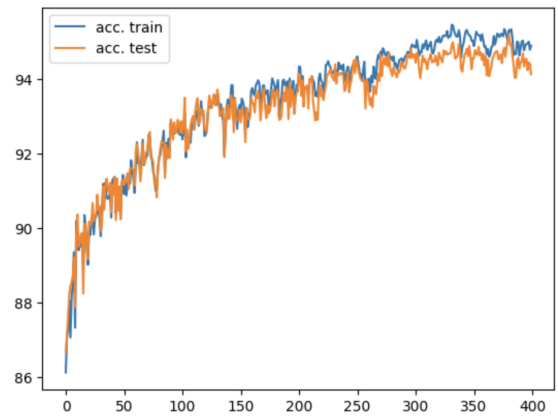
To conclude on this part, using the libraries all ready implemented in pytorch isn't always the best solution. Even though it is computationally more efficient, the models can perform way better when forward or backward methods are implemented manually.

3.5 MNIST Application

For this part I used the last method because even though it wasn't the one that performed the best, it is the most commonly used and it is also very fast to compute (especially for a bigger dataset such as the MNIST dataset).



(a) Loss function of our model on the MNIST dataset



(b) Accuracy of our model on the MNIST dataset

Figure 17: Training and test performances on the MNIST dataset

The FIGURE 17A and FIGURE 17B show pretty common accuracy and loss function with a little bit of instability (that could be corrected by changing for example the learning rate or the batch size as we saw earlier).

The model is able to reach an accuracy of 95% after 400 iterations which takes approximately 15 minutes to run.

```
[[ 958    0    3    2    0    5    9    1    0    2]
 [    0 1110    4    3    0    2    4    4    8    0]
 [    5    4  980   10    8    3    4    8   10    0]
 [    2    0   14  928    2   16    1   12   26    9]
 [    2    2    3    0  920    0   21    2    5   27]
 [   11    0    4   25    4  810   13    2   13   10]
 [   11    1    0    1   12    7  921    0    5    0]
 [    4    8   23    8    3    1    0  949    4   28]
 [    7    4    7   12    6    6    8    9  908    7]
 [   11    6    1   12   26    4    0    9   11  929]]
```

Figure 18: Confusion matrix of the multi-classification of the dataset MNIST

The FIGURE 18 represents the amount of images well classified (on the diagonal) and also those misclassified and where they have been put. It enables us to see what classes look similar for example 26 images of the 4th class have been classified in

the 9th but 0 of them were classified in the 2nd class. This shows relations between classes and can help improve the model by (for example) trying to add extra loss for similar classes to better differentiate them.